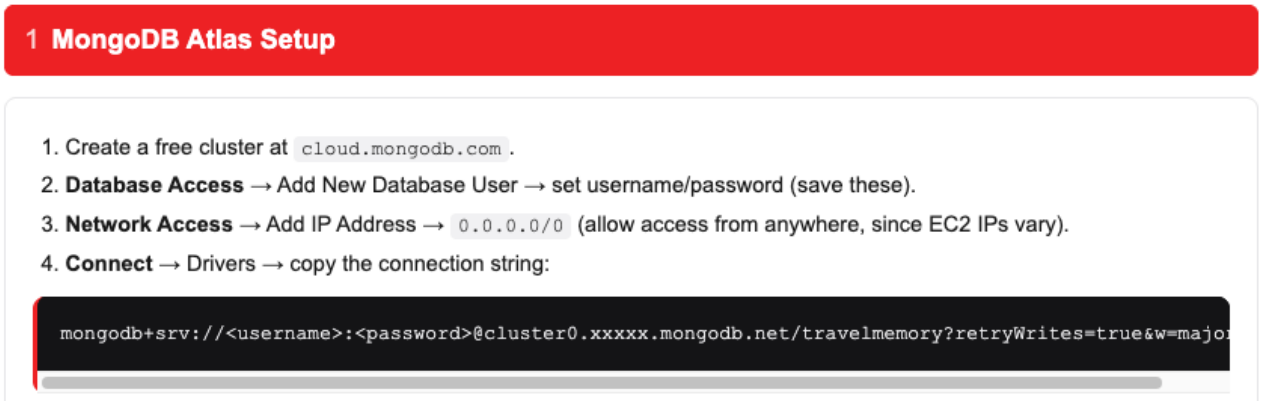
MongoDB Atlas Setup Guide

Step-by-step: cluster creation, connection string and database user password

Reference: Setup instructions

The setup follows the guideline below (MongoDB Atlas Setup, for the travelmemory app):



Screenshot: Reference instructions

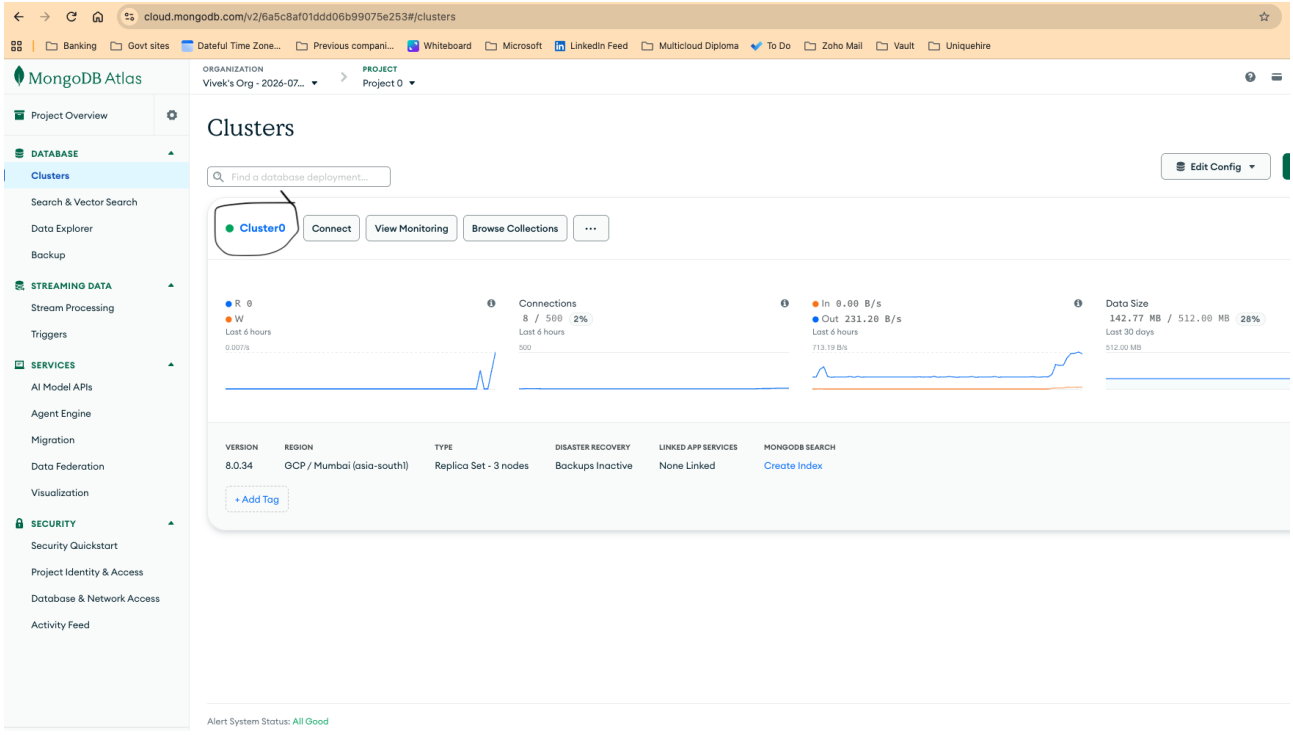
Summary of the guideline: create a free cluster at cloud.mongodb.com; add a database user with username/password; allow network access from 0.0.0.0/0; then use Connect → Drivers to copy the connection string.

Step 1: Cluster0 created in MongoDB Atlas

A free cluster named **Cluster0** was created at cloud.mongodb.com under Organization "Vivek's Org", Project 0. The Clusters page (Database → Clusters) shows it as active (green dot).

Cluster details:

- Cluster name: Cluster0
- Version: 8.0.34
- Cloud / Region: GCP / Mumbai (asia-south1)
- Type: Replica Set - 3 nodes
- Storage used: 142.77 MB of 512 MB



Screenshot: Clusters page showing Cluster0 (circled) and the Connect button

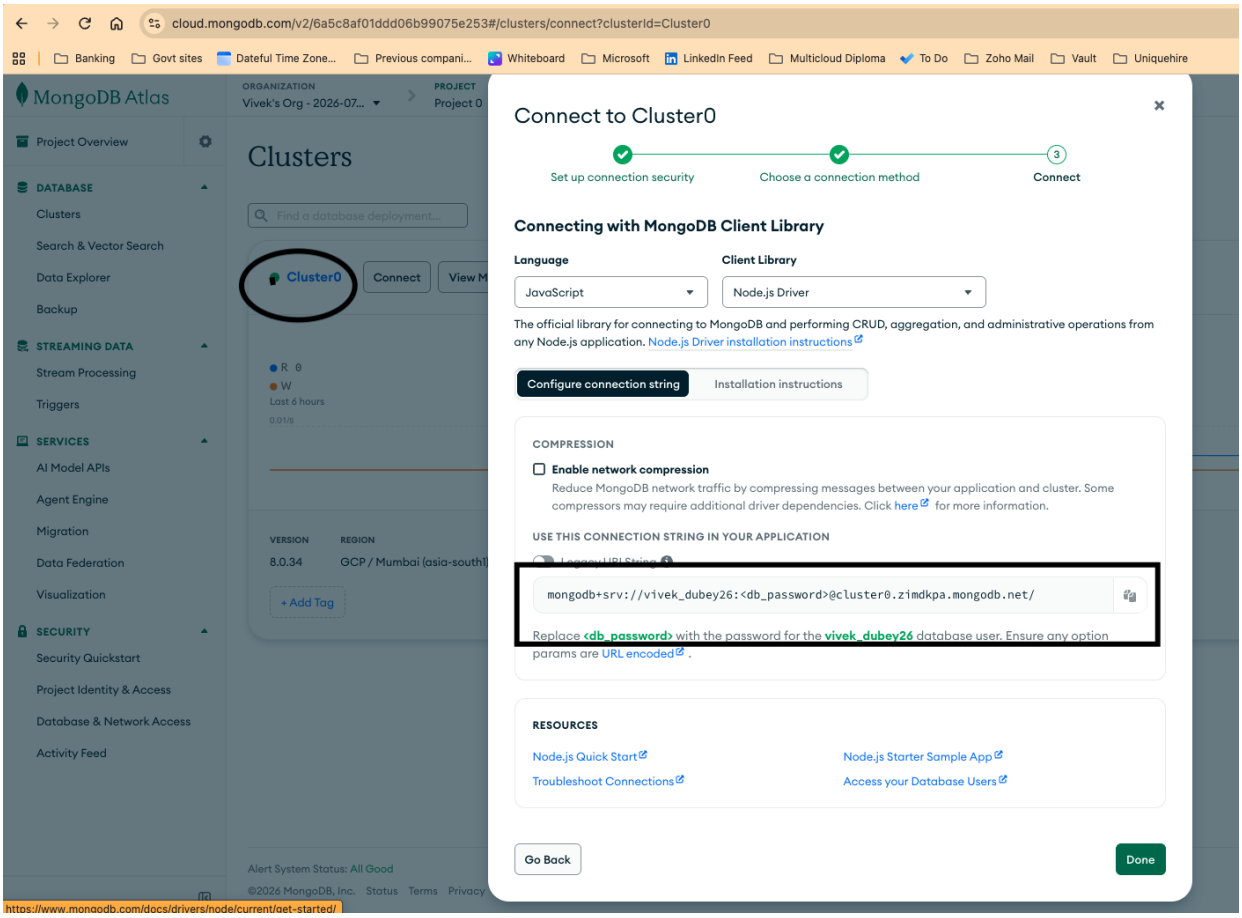
Step 2: Copy the connection string

1. On the Clusters page, click **Connect** next to Cluster0.

2. Complete the connection security setup, then choose the connection method **Drivers**.

3. Select Language: **JavaScript** and Client Library: **Node.js Driver**.

4. Under 'Use this connection string in your application', click the copy icon next to the string (highlighted in the screenshot).



Screenshot: Connect to Cluster0 dialog with the connection string highlighted

Connection string copied:

```
mongodb+srv://vivek_dubey26:<db_password>@cluster0.zimdkpa.mongodb.net/
```

Step 3: Set the password for user vivek_dubey26

The copied string contains the placeholder **<db_password>**. As the dialog says, replace it with the password of the database user **vivek_dubey26**. The password was changed for this user (Database Access) and then substituted into the string.

To change the password in Atlas:

1. Go to **Security** → **Database & Network Access** (left menu).
2. Open the **Database Access** tab and click **Edit** next to vivek_dubey26.
3. Choose **Edit Password**, enter the new password (or use Autogenerate) and save the user.
4. Replace **<db_password>** in the connection string with the new password.

Final connection string format:

```
mongodb+srv://vivek_dubey26:<NEW_PASSWORD>@cluster0.zimdkpa.mongodb.net/travelmemory?retryWrites=tr
```

Notes

• Remove the angle brackets < > when inserting the password.

• If the password has special characters (@ : / ? # %), URL-encode them (e.g. @ becomes %40).

• Per the reference guide, add the database name (travelmemory) after the host, before the '?'.

• Network Access must allow 0.0.0.0/0 so EC2 instances with changing IPs can connect.

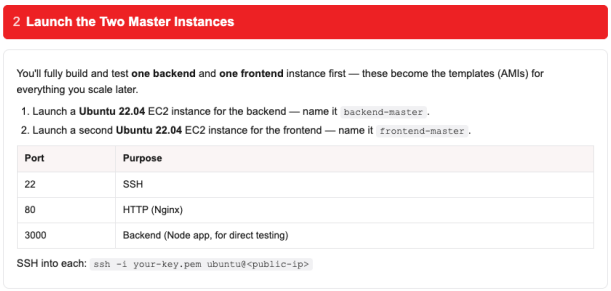
• Never share the real password or commit it to GitHub; keep it in a .env file.

Launch the Two Master Instances

AWS EC2 – backend-master and frontend-master (Ubuntu 22.04)

Objective. Fully build and test **one backend** and **one frontend** instance first. These two instances become the templates (AMIs) for everything that is scaled later.

Reference: instruction screenshot



2 Launch the Two Master Instances

You'll fully build and test **one backend** and **one frontend** instance first — these become the templates (AMIs) for everything you scale later.

1. Launch a **Ubuntu 22.04** EC2 instance for the backend — name it `backend-master`.
2. Launch a second **Ubuntu 22.04** EC2 instance for the frontend — name it `frontend-master`.

Port	Purpose
22	SSH
80	HTTP (Nginx)
3000	Backend (Node app, for direct testing)

SSH into each: `ssh -i your-key.pem ubuntu@<public-ip>`

Figure 1 – Task instructions

Step 1: Launch the backend instance

• Open the AWS Console, switch to the **Asia Pacific (Mumbai) ap-south-1** region, and go to **EC2** → **Instances** → **Launch instances**.

• Set the **Name** tag to `backend-master`.

• Choose the **Ubuntu Server 22.04 LTS** AMI and instance type **t2.micro**.

• Select your key pair (the `.pem` file used later for SSH).

• Under network settings, create a security group that allows ports 22, 80 and 3000 (see Step 3), then click **Launch instance**.

Step 2: Launch the frontend instance

• Click **Launch instances** again.

• Set the **Name** tag to `frontend-master`.

• Use the same Ubuntu 22.04 AMI, t2.micro type, key pair and security group settings, then launch.

Both instances are now visible in the EC2 Instances list. Their state is **Running** and status checks show **2/2 checks passed**. The highlighted rows below are the two master instances (backend-master in ap-south-1a, frontend-master in ap-south-1b).

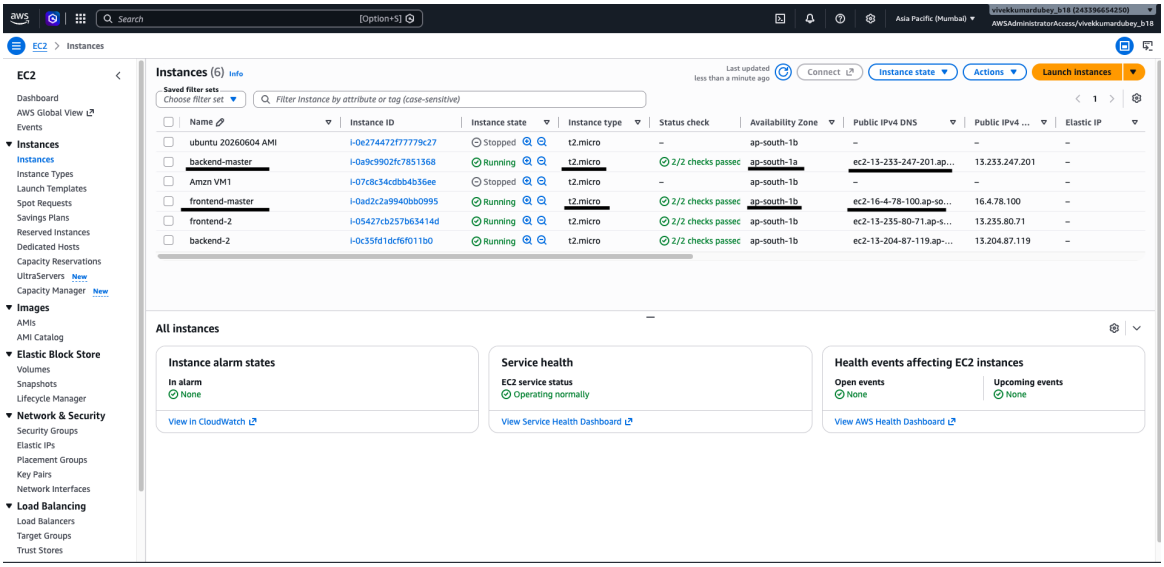


Figure 2 – backend-master and frontend-master running in EC2

Step 3: Open the required ports in the Security Group

The inbound rules of the security group must allow the following ports:

Port	Purpose
22	SSH
80	HTTP (Nginx)
3000	Backend (Node app, for direct testing)

To verify: select the instance → **Security** tab → **Inbound rules**. Here the security group `sg-0d626c26180b83acd (launch-wizard-4)` shows TCP rules for ports **22, 80, 3000** (and additionally 443 for HTTPS), source `0.0.0.0/0`.

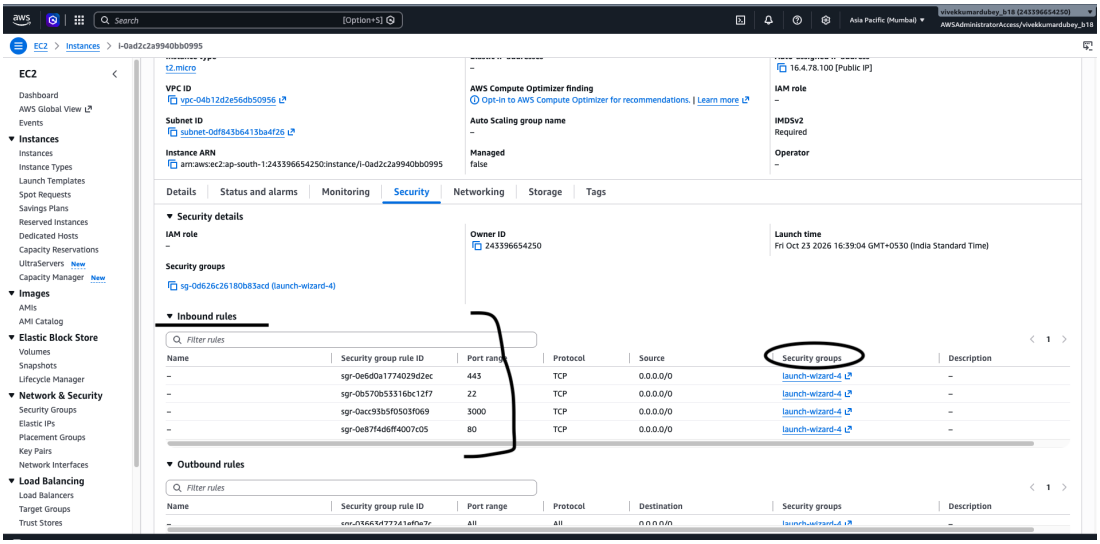


Figure 3 – Inbound rules on the master instance security group

Note: Source 0.0.0.0/0 allows access from anywhere. This is acceptable for a learning setup, but in production restrict SSH (22) to your own IP and port 3000 to trusted sources only.

Step 4: SSH into each instance

Copy the **Public IPv4 address** of each instance from the EC2 console and connect from your terminal:

```
ssh -i your-key.pem ubuntu@<public-ip>
```

Instance	Public IPv4	Command
backend-master	13.233.247.201	`ssh -i your-key.pem ubuntu@13.233.247.201`
frontend-master	16.4.78.100	`ssh -i your-key.pem ubuntu@16.4.78.100`

If SSH reports *permissions too open*, run `chmod 400 your-key.pem` first. Type **yes** at the host fingerprint prompt on first connect.

Next

With both masters reachable, install and test the Node backend (port 3000) and the Nginx frontend (port 80). Once verified, create AMIs from these instances to scale later.

Backend and Frontend Setup

TravelMemory on AWS EC2: Step-by-Step Guide with Verification Screenshots

Overview

This document walks through setting up the two master instances: **backend-master** (Node.js API run as a systemd service behind Nginx) and **frontend-master** (React build served by Nginx). Both run Ubuntu in region Asia Pacific (Mumbai). Screenshots from the actual instances are included under the relevant steps.

Step 3: Backend Setup (on backend-master)

3.1 Install dependencies and clone the repository

```
sudo apt update
sudo apt install -y nodejs npm git nginx
git clone https://github.com/UnpredictablePrashant/TravelMemory.git
cd TravelMemory/backend
npm install
```

3.2 Create the environment file at `backend/.env`

```
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/
travelmemory?retryWrites=true&w=majority
PORT=3000
```

3.3 Test the app. Run `node index.js`, then from another terminal run `curl localhost:3000/hello`. Stop with Ctrl+C once confirmed.

3.4 Run the backend as a systemd service (no PM2)

A systemd service keeps the app running after SSH closes, restarts it on crash, and starts it automatically on boot. This matters because every instance launched later from the AMI must bring the backend up on its own.

Create `/etc/systemd/system/travelmemory-backend.service`:

```
[Unit]
Description=TravelMemory Backend
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/TravelMemory/backend
EnvironmentFile=/home/ubuntu/TravelMemory/backend/.env
ExecStart=/usr/bin/node index.js
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now travelmemory-backend
sudo systemctl status travelmemory-backend
```

3.5 Nginx reverse proxy

Edit `/etc/nginx/sites-available/default` and replace the default `location /` block with:

```
location / {
    proxy_pass http://localhost:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

```
sudo nginx -t
sudo systemctl restart nginx
```

3.6 Verify: `curl http://<backend-master-public-ip>/hello` should return `Hello World!`

3.7 Verification screenshot: backend-master

The terminal on backend-master (instance `i-0a9c9902fc7851368`, current public IP 13.233.247.201 (it was 13.127.213.50 at setup time), private IP 172.31.35.43) shows: Node.js version output, the **travelmemory-backend** service **enabled** and **active (running)** (`node index.js`, "Server started at http://localhost:3000"), and the **nginx** service **enabled** and **active (running)**.

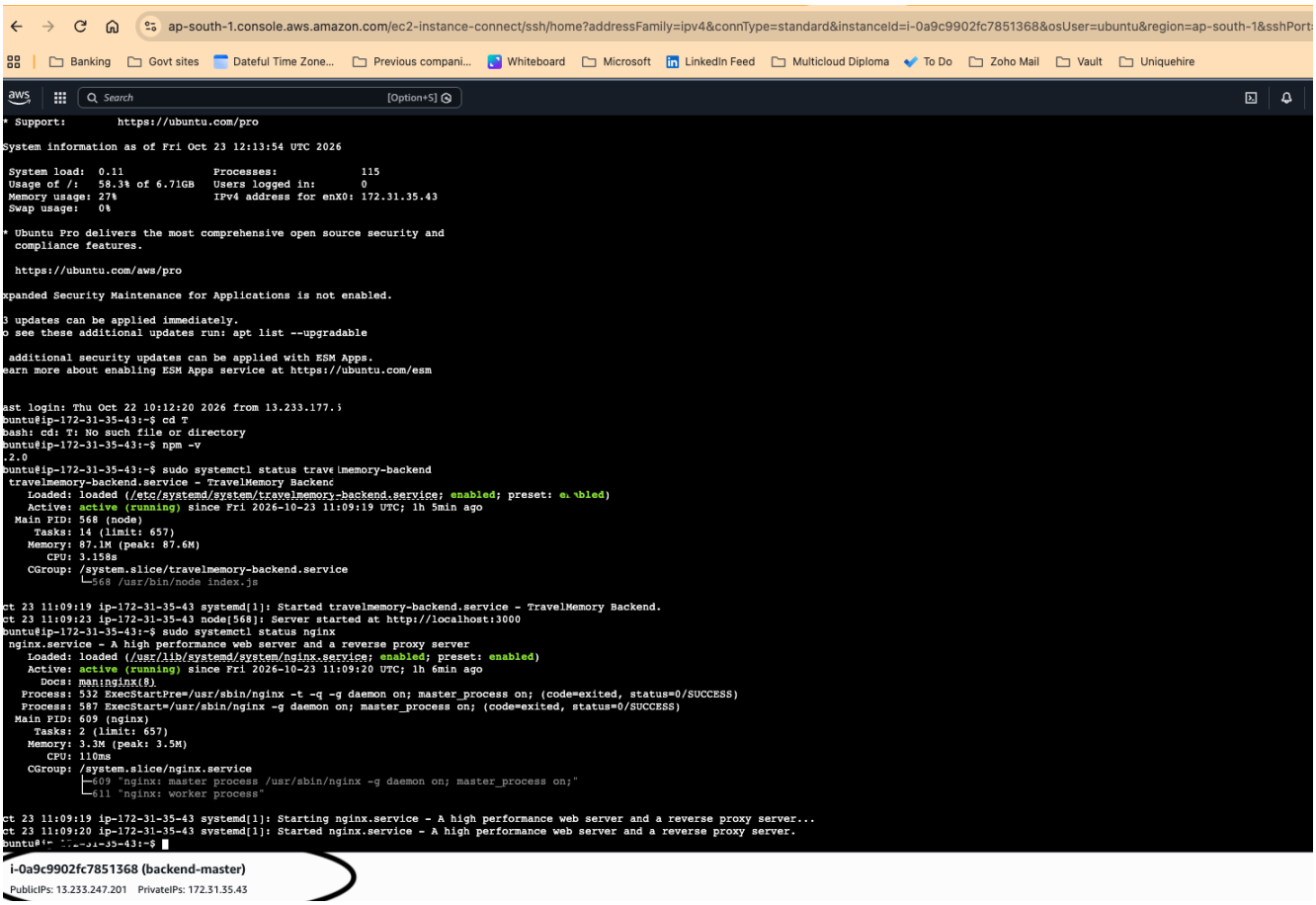


Figure 1: backend-master terminal showing npm/node version, systemctl status travelmemory-backend and systemctl status nginx.

Step 4: Frontend Setup (on frontend-master)

4.1 Install dependencies and clone the repository

```
sudo apt update
sudo apt install -y nodejs npm git nginx
git clone https://github.com/UnpredictablePrashant/TravelMemory.git
cd TravelMemory/frontend
```

4.2 Point the frontend at the backend. Open `src/url.js` and set `baseUrl` to the backend master public IP. This is an interim value and will be repointed to the backend load balancer in Step 10.

```
export const baseUrl = "http://<backend-master-public-ip>";
```

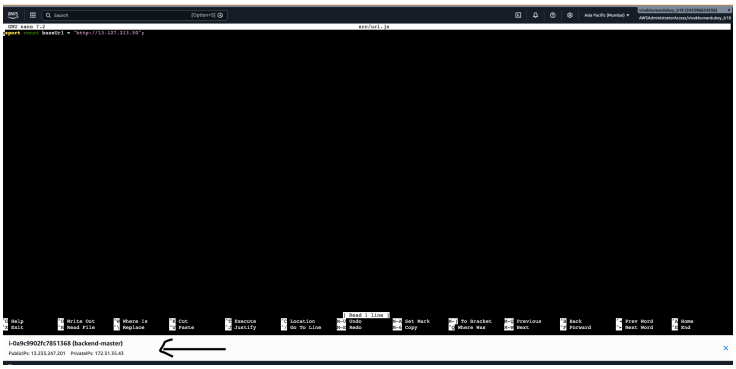


Figure 2: src/url.js opened in nano on frontend-master. The footer shows the current public IP of backend-master, which changed after a restart (see note below).

Check before moving on: in Figure 2 the `baseUrl` is `http://13.127.213.50`, which was the backend-master public IP at the time the backend was set up. Every time an EC2 instance reboots (stops and starts), it is assigned a new public IP unless an Elastic IP is attached. The footer now shows a different public IP, `13.233.247.201`, so the frontend still points at the old address. Before verifying, update `src/url.js` to the current backend-master public IP, rebuild, and redeploy. This is also why the value is only interim until the load balancer DNS name replaces it in Step 10.

4.3 Build and deploy to Nginx

```
npm install
npm run build
sudo rm -rf /var/www/html/*
sudo cp -r build/* /var/www/html/
sudo systemctl restart nginx
```

Step 5: Verify the Baseline Before Imaging

Open `http://<frontend-master-public-ip>` in a browser. Confirm that trips load and that adding a new experience works end to end (frontend, backend, Atlas).

Do this before creating any AMI. Anything broken now is copied into every instance launched from the image.

TravelMemory – Load Balancer Setup Evidence

Steps 8–10: Target Groups, Application Load Balancers and pointing the frontend at the backend ALB. Region: Asia Pacific (Mumbai).

Step 8 – Backend Target Group + Load Balancer

Target group **travelmemory-backend-tg** (HTTP, port 80, health check `/hello`) with backend-master and backend-2 registered, fronted by the internet-facing ALB **travelmemory-backend-alb**.

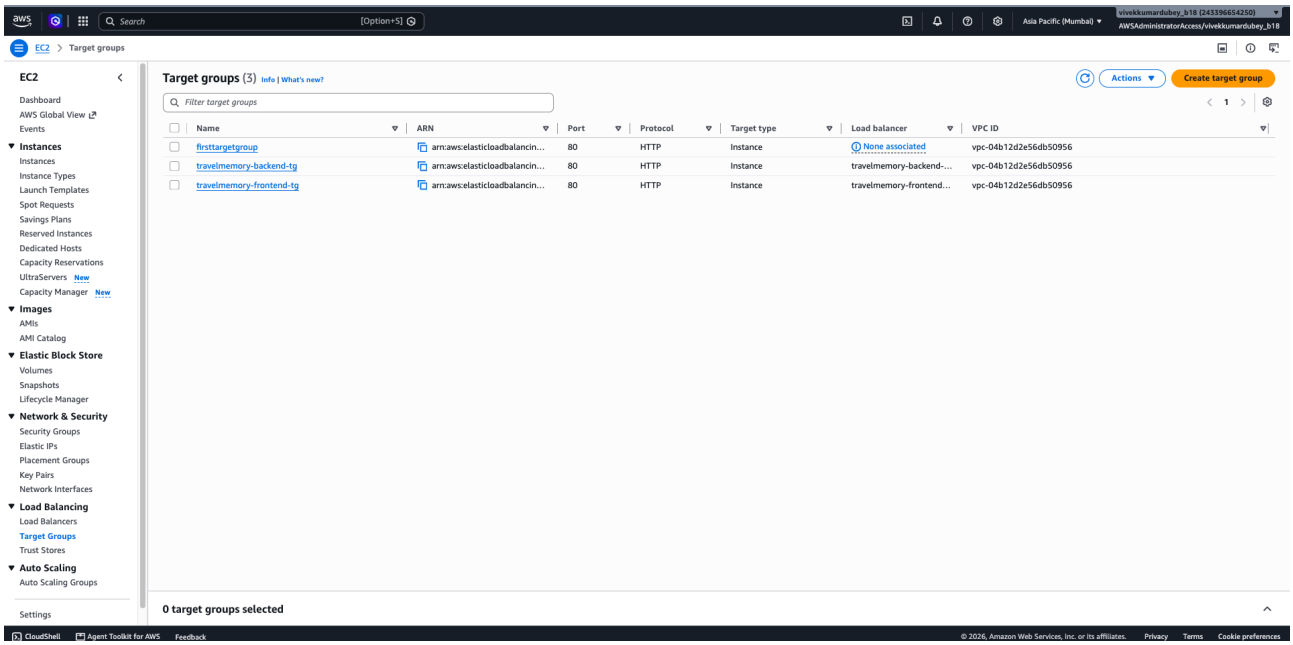


Figure 1: EC2 → Target Groups. travelmemory-backend-tg and travelmemory-frontend-tg are created (HTTP, port 80, Instance type) and attached to their load balancers.

Step 9 – Frontend Target Group + Load Balancer

Target group **travelmemory-frontend-tg** (HTTP, port 80, health check `/`) with frontend-master and frontend-2 registered, fronted by the internet-facing ALB **travelmemory-frontend-alb**.

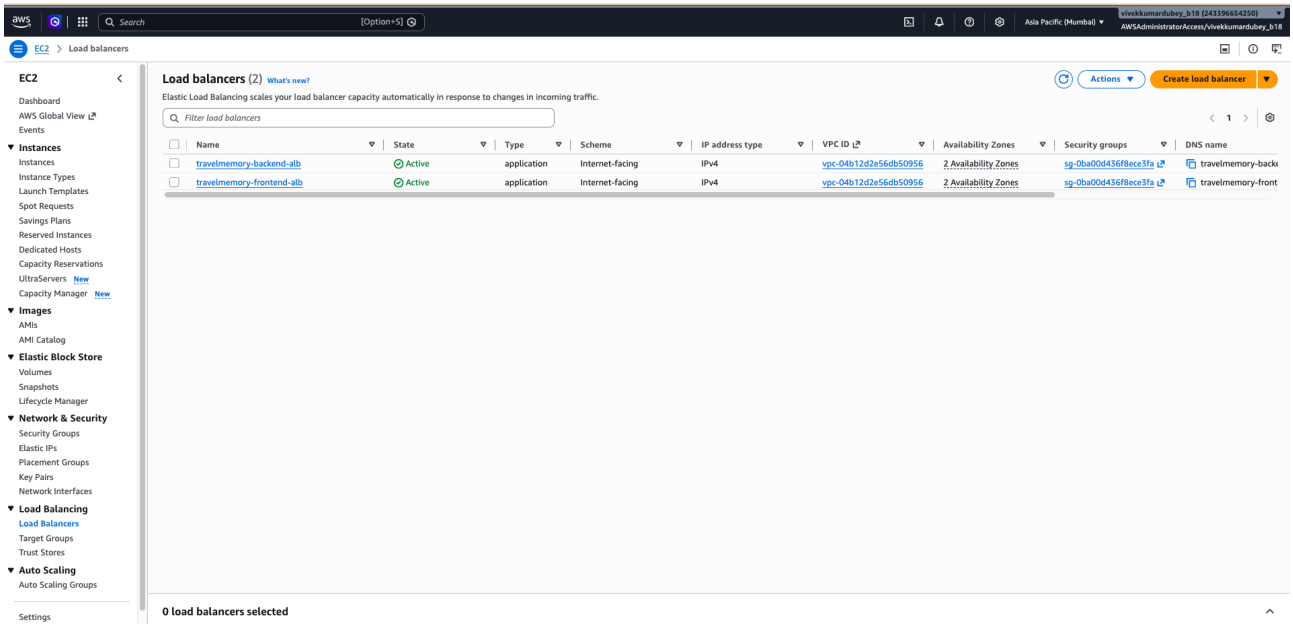


Figure 2: EC2 → Load Balancers. travelmemory-backend-alb and travelmemory-frontend-alb are both Active, internet-facing, application type, across 2 Availability Zones.

Step 10 – Point Frontend at the Backend Load Balancer

On both frontend instances, `src/url.js` was edited so that `baseUrl` uses the backend ALB DNS name, followed by `npm run build`, copying the build to `/var/www/html/` and restarting nginx.

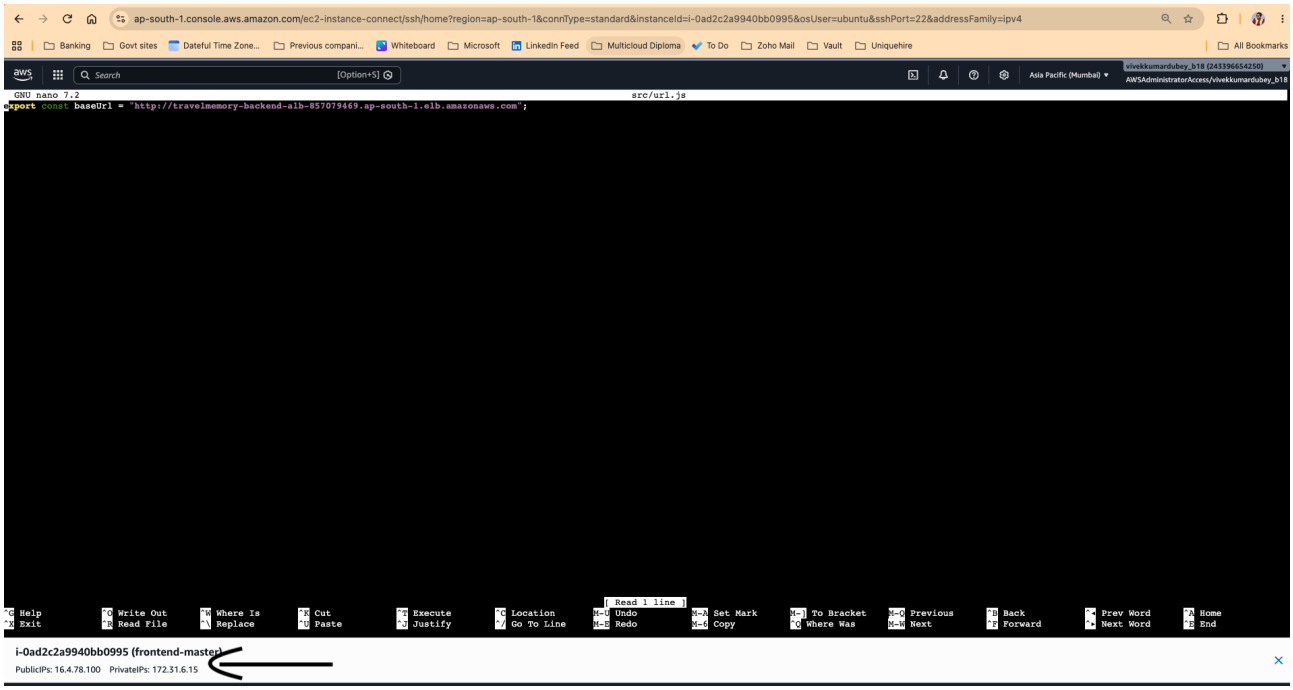


Figure 3: frontend-master (i-0ad2c2a9940bb0995) – src/url.js points to the backend ALB.

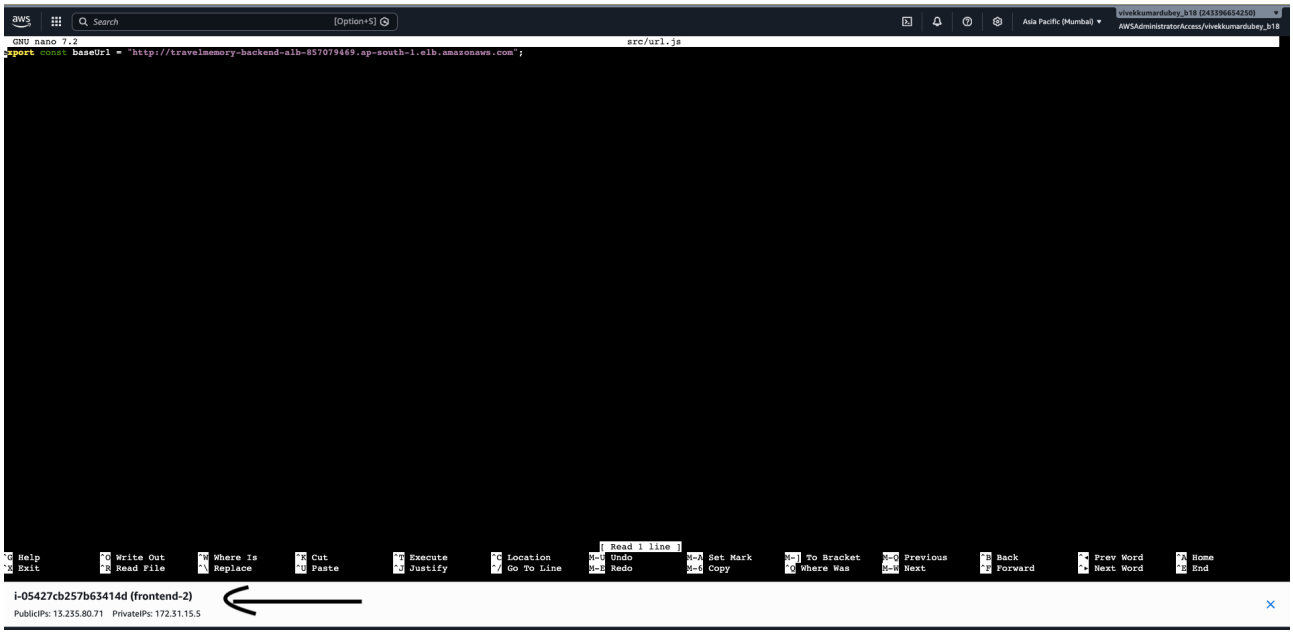


Figure 4: frontend-2 (i-05427cb257b63414d) – src/url.js points to the backend ALB.

Verification

Testing the application end to end. The first two screenshots show an entry added before the switch (served from an instance IP); the last two show the application served through the frontend ALB, with the new entry stored in MongoDB.

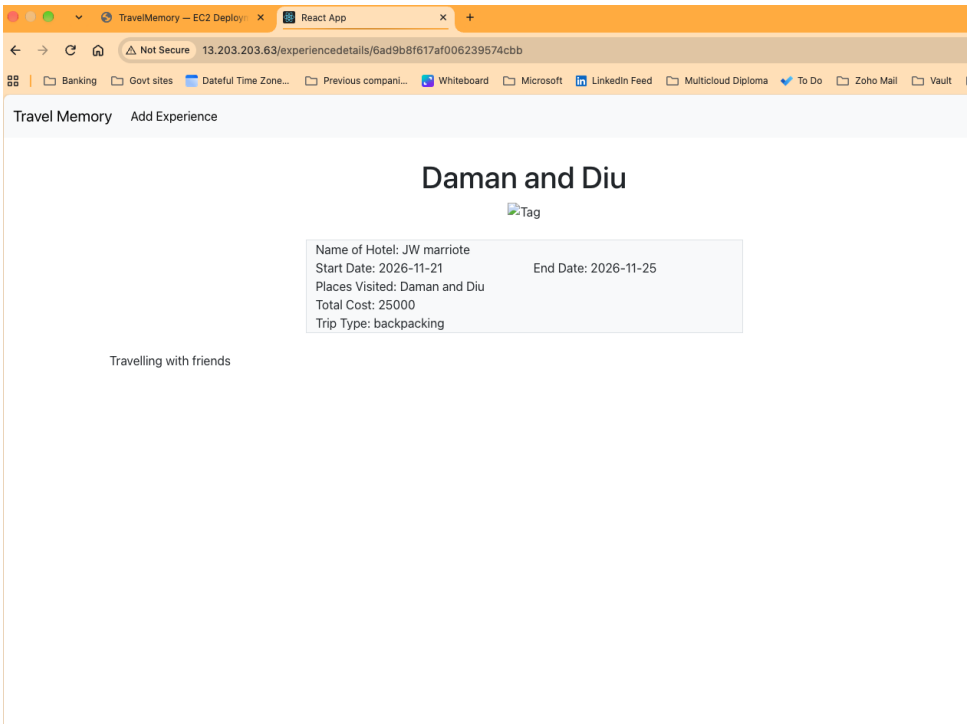


Figure 5: Experience page (Daman and Diu) loaded from the frontend instance.

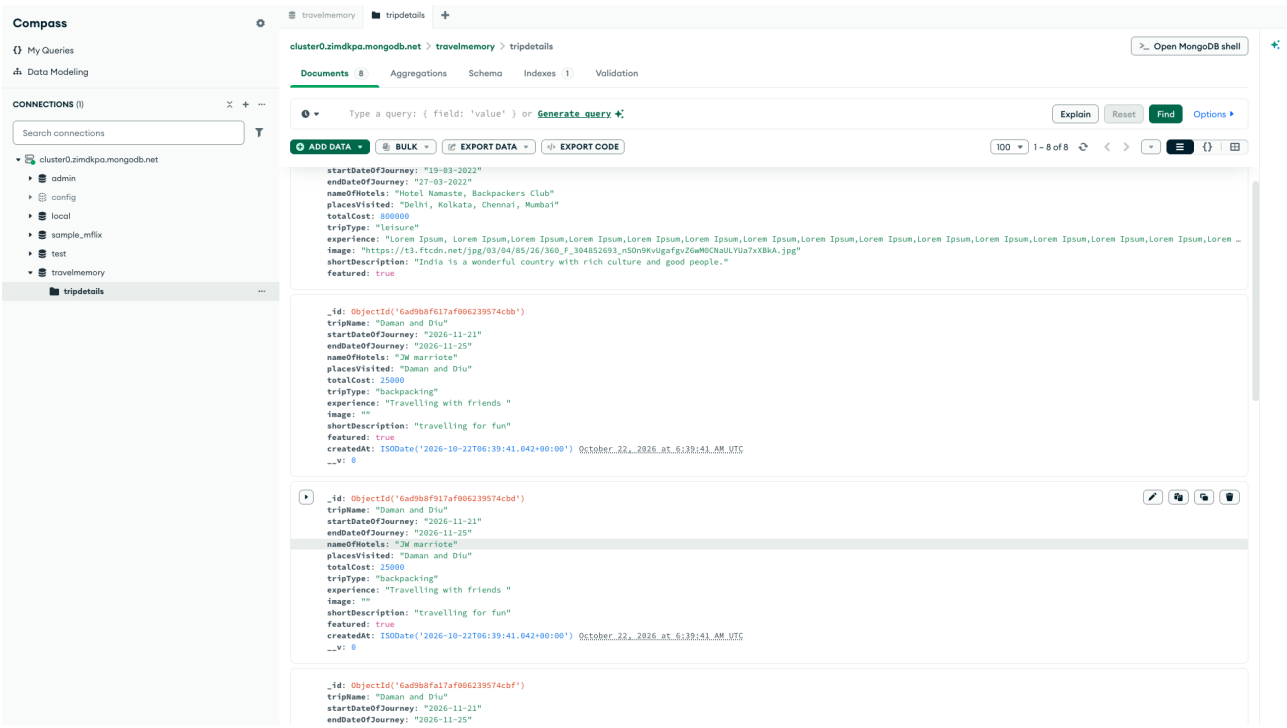


Figure 6: MongoDB Compass – travelmemory.tripdetails collection holding the saved trip documents.

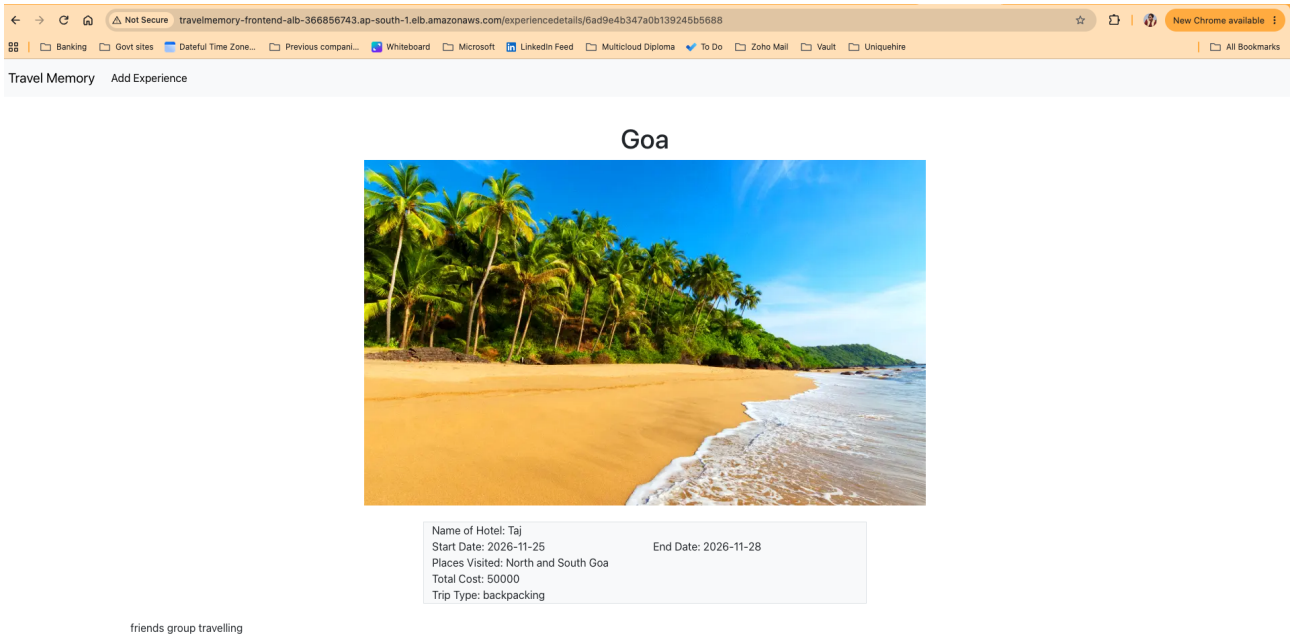


Figure 7: Application opened via the frontend ALB DNS (travelmemory-frontend-alb-366856743.ap-south-1.elb.amazonaws.com) showing the “Goa” trip details.

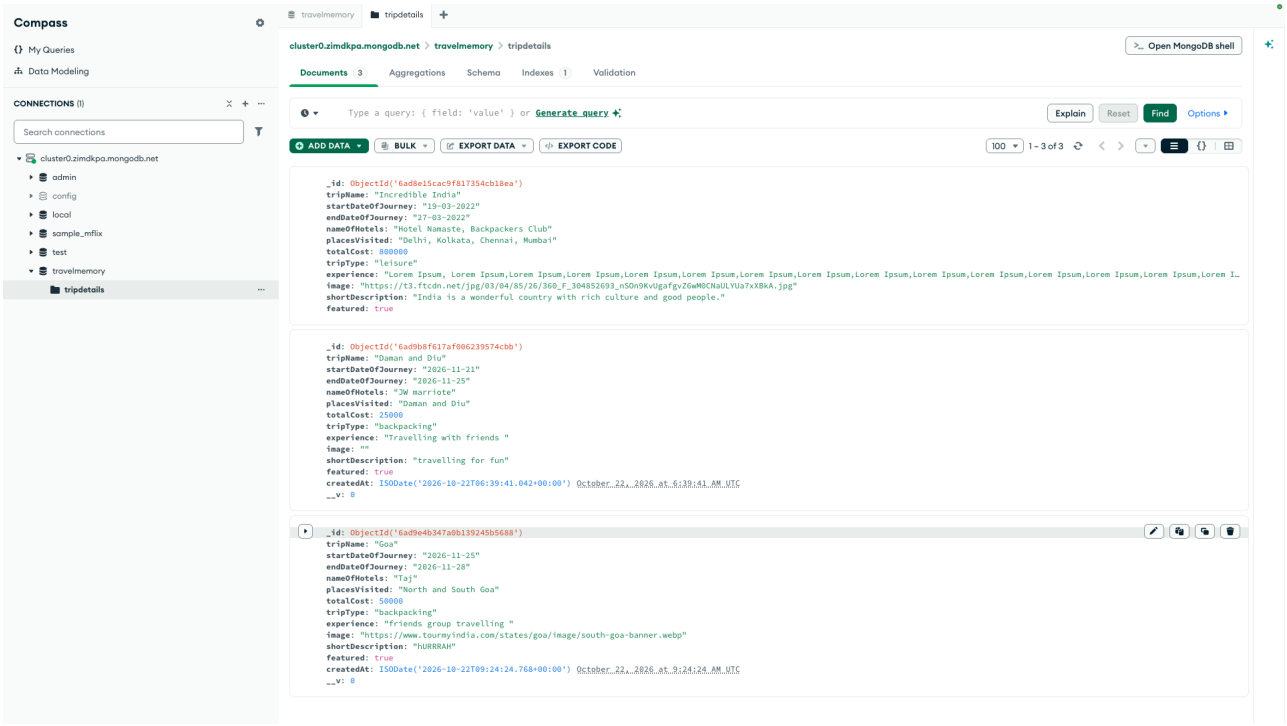


Figure 8: MongoDB Compass – the “Goa” document created through the ALB-fronted app is present in tripdetails.

Added records on GoDaddy and Hosted the application on www.migratedmonk.in

DNS records were added in GoDaddy so the domain points to the load balancers (CNAME **api** to the backend ALB, CNAME **www** to the frontend ALB), and the application is now served on the domain.

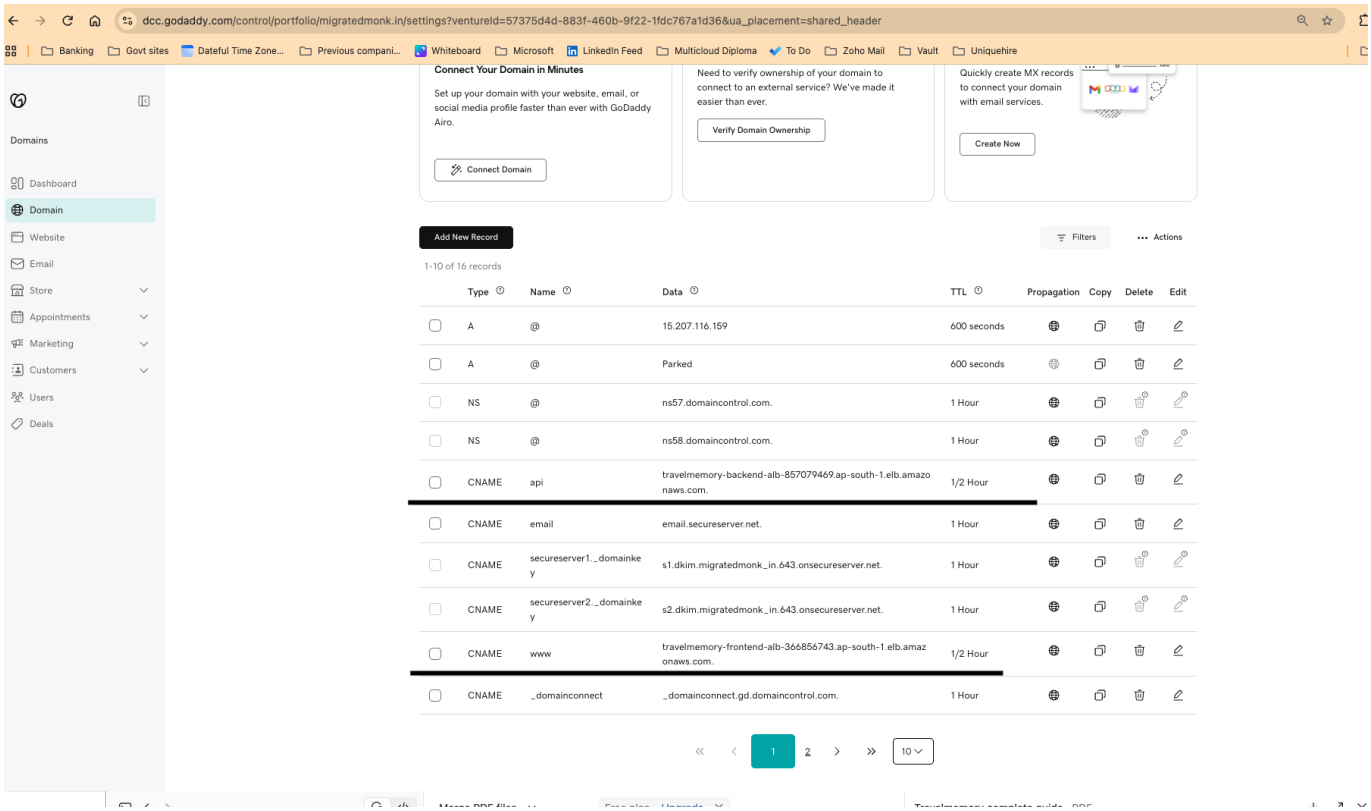


Figure 1: GoDaddy DNS settings for migratedmonk.in. The highlighted CNAME records are api (backend ALB) and www (frontend ALB).

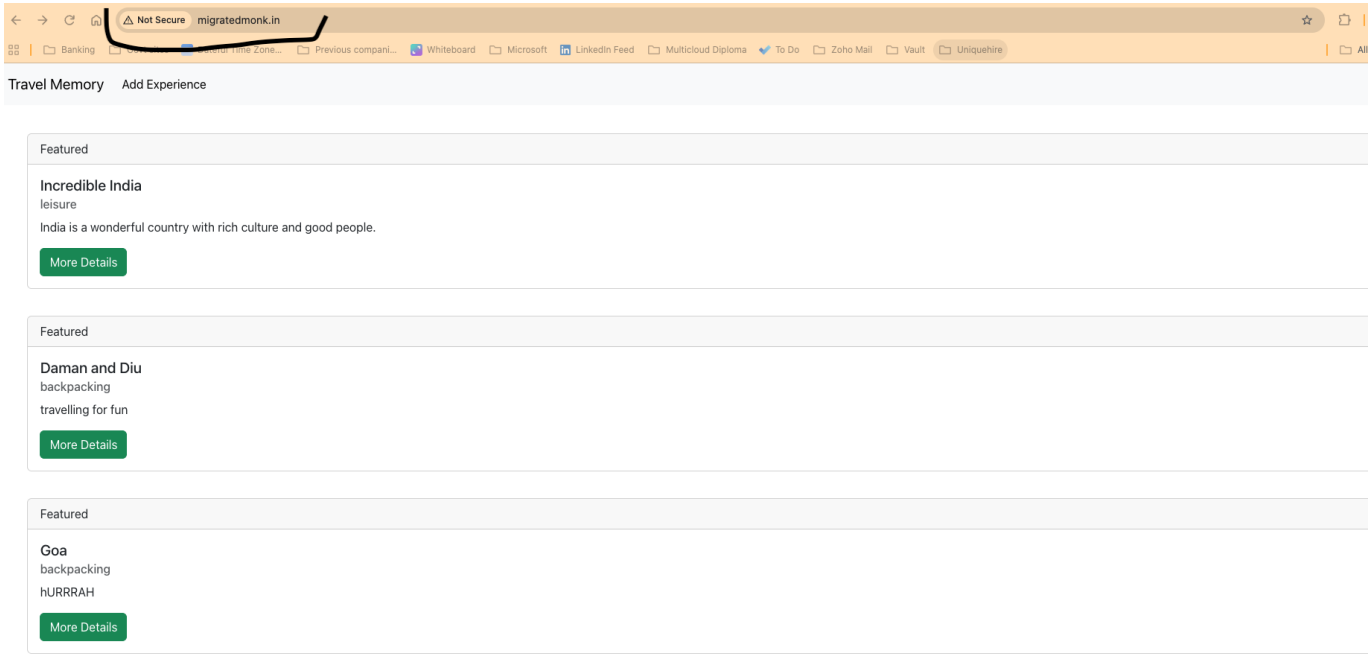


Figure 2: TravelMemory application opened at migratedmonk.in, showing the trips Incredible India, Daman and Diu, and Goa.